Embeddings as a Bridge Between Neural and Classical Recommendation Models

Shivanshu Sirohi · May 2026 · Machine Learning, Recommender Systems · ~15 min read

Here is a question that sounds simple but turns out to be hard to answer rigorously: if you train a neural network to learn dense representations of categorical features, can other models benefit from those representations? Classical models like XGBoost or Linear Regression?

Intuitively it seems like it should work. The neural network has learned something meaningful about the structure of the data. Why not let everyone else use it too?

I ran a systematic experiment across three datasets and four algorithms to find out. The short answer is **yes**; **embeddings do transfer**. But the benefit is deeply heterogeneous across model classes, and the reasons why tell you something fundamental about what embeddings encode.

The Central Question

The study is built around one research question: do ANN-learned categorical embeddings transfer effectively to classical recommendation models, and what determines how much benefit each model class receives?

To answer this cleanly, I designed a three-condition experiment:

Experiment 1 (Baseline). One-hot encoding and multi-hot encoding for all categorical features. Default hyperparameters. The standard starting point most practitioners use.

Experiment 2 (Embeddings). An Embedding ANN learns dense vector representations for all categoricals. Those embedding weight matrices are extracted and used as feature columns by Linear Regression, Random Forest, and XGBoost. Hyperparameters stay at default, isolating the encoding effect.

Experiment 3 (Optimised). Optuna TPE search tunes hyperparameters on the embedding features. This isolates the tuning contribution separately from encoding.

The three-condition structure is deliberate. A two-condition study comparing baseline against optimised cannot separate encoding strategy from hyperparameter tuning. The intermediate condition is what makes the results interpretable.

One framing clarification before we go further. For Linear Regression, Random Forest, and XGBoost, Experiment 2 is genuine cross model transfer: they receive pre-computed embedding weights as static feature columns, produced by a different architecture. For the ANN, it is different. The ANN already learns embeddings internally. Experiment 2 for the ANN is representation substitution: replacing sparse OHE inputs with learned dense inputs. The benefit comes from compactness and information density, not from cross architecture transfer.

Four algorithms, three datasets, all splits temporal (train on past, predict on future).

The Three Datasets

Each dataset was chosen to create a different challenge for the embedding approach.

MovieLens 20M (100K temporally sampled subset). The standard recommendation benchmark. Rich external metadata from IMDB and TMDB: ratings, vote counts, budget, revenue, genres, language. Ratings approximately normally distributed around 3.5 stars. The categoricals here are low cardinality and semantically meaningful.

Amazon Product Reviews (500K sample). Electronics reviews with no product metadata. The only categoricals are user IDs and product IDs, both high cardinality and semantically opaque. 55% of all ratings are 5-star. Everything must come from interaction history alone.

Yelp Academic Dataset (100K sample). Restaurant and business reviews. Introduces a third type of categorical: city, state, and business categories (comma separated multi-label, similar to MovieLens genres). Medium cardinality. Integer 1–5-star ratings.

The progression from metadata rich to interaction only to geographically structured tests whether embedding transfer is a general phenomenon or dataset specific.

A Critical Engineering Detail

For Amazon and Yelp, where no external metadata exists, all features must come from interaction history. This requires Leave-One-Out (LOO) aggregation. It is a detail that cost me two days to debug properly.

The naive approach computes a user's average rating across all their training reviews and merges it back onto those same rows. For a row (User A, Product X, rating=5), product_avg_rating already contains that 5. The model is not learning. It is looking up the answer.

My initial Amazon version had Train RMSE 0.04 and Test RMSE 1.70. The fix is simple, but the bug is non-obvious:

```
# Wrong — this row's own rating is in the average
user_avg = df_train.groupby('userId')['rating'].mean()

# Correct — Leave-One-Out: exclude this row user_avg
= np.where((user_count - 1) > 0,(user_sum -
this_rating) / (user_count - 1),np.nan

# single-review users get global average )
```

The np.where matters here. Using .clip(lower=1) to avoid division by zero produces 0.0 for single review users. That 0.0 is not NaN, so fillna never fires, and you end up with user_avg_rating showing a mean of 1.5 instead of 4.0. The feature looks computed. It is silently wrong for hundreds of thousands of rows.

What the Experiments Showed

MovieLens: The Feature Ceiling

MovieLens produced the most modest results of the three datasets. Every improvement was small:

Model	Baseline	Embeddings	Optimised	Total Change
Linear Regression	0.9381	0.9293	0.9293	-0.0088
Random Forest	0.9641	0.9340	0.9279	-0.0362
XGBoost	0.9394	0.9319	0.9274	-0.0120
ANN	0.9338	0.9333	0.9300	-0.0038

Table 1: MovieLens Test RMSE. Lower is better.

The total range across all conditions and all algorithms was 0.006 to 0.036 RMSE. XGBoost won at 0.9274, but it barely edged out the competition.

This is the feature ceiling effect. MovieLens models already know a movie's IMDB rating, box office budget, and genre. Once that information is in the feature set, encoding strategy and

algorithm choice have diminishing returns. The ceiling is set by the available information, not by how it is processed.

One observation worth noting: on MovieLens, embedding substitution provided more improvement than hyperparameter tuning for most models. This pattern reverses on Amazon.

Amazon: Where Embeddings Earned Their Keep

Amazon sits at the opposite end of the spectrum from MovieLens. No product metadata. The only categoricals are high cardinality IDs. The embedding experiment required careful engineering.

The first attempt produced validation RMSE that went from 1.86 at epoch 1 to 7.50 at epoch 20. With 400,000+ unique users and 32 embedding dimensions, the embedding table contained 13 million parameters against 350,000 training rows. The model memorised user specific patterns rather than learning generalisable preference signals.

Four fixes resolved this: vocabulary filtering (only embed entities with 5+ reviews), reduced embedding dimensions (32 to 8), dropout on the embedding layer (0.5), and early stopping with patience of 5 epochs.

Model	Baseline	Embeddings	Optimised	Total Change
Linear Regression	1.3646	1.3645	1.3644	-0.0002
Random Forest	1.5559	1.5153	1.3626	-0.1933
XGBoost	1.4942	1.4764	1.3579	-0.1363
ANN	1.4746	1.3971	1.3729	-0.1017

Table 2: Amazon Test RMSE.

The total improvement range here is 0.1933. That is ten times larger than MovieLens. Random Forest went from 1.5559 to 1.3626, almost entirely because Optuna found appropriate depth constraints for the trees. The default max_depth=None setting had been causing catastrophic memorisation.

Linear Regression barely moved (1.3646 to 1.3644 across all three experiments). This number matters for the central argument.

Yelp: The Surprise

Yelp introduced high cardinality OHE. With 758 unique cities and 1,166 category tokens, the baseline feature matrix had approximately 1,944 columns. The ANN showed a Train-Test gap of 1.13, the worst overfitting in the entire study.

Model	Baseline	Embeddings	Optimised	Best Result
Linear Regression	1.5672	1.5526	1.5542	1.5526
Random Forest	1.5831	1.7030	1.5820	1.5820
XGBoost	1.6517	1.6716	1.5597	1.5597
ANN	1.9168	1.5869	1.5871	1.5869

Table 3: Yelp Test RMSE.

Two things stand out. The ANN improved by 0.3299 from baseline to embeddings. Replacing the 1,944 column OHE matrix with 58 dense embedding features eliminated the overfitting entirely. That is the largest single improvement in the whole study.

The second observation: Linear Regression won overall at 1.5526 (from Experiment 2). On 70,000 training rows, the simplest well regularised model generalised best. This contrasts with MovieLens and Amazon, where XGBoost won. Dataset size appears to influence which model class handles the bias-variance tradeoff most effectively.

> **"ANN-learned embeddings transfer to classical models, but the magnitude of benefit is determined by how badly the alternative encoding was causing overfitting and how well the model can act on relational structure in embedding space."**

Why the Benefit Differs Across Model Classes

Linear Regression: small but consistent improvement

Across all three datasets, LR improvement from embeddings was small: 0.0088 on MovieLens, 0.0001 on Amazon, 0.0146 on Yelp. These are real improvements, but consistently much smaller than what the ANN gained on the same datasets.

The reason is that embedding space encodes relational proximity. Italian restaurants cluster near French restaurants because users who like one tend to like the other. Electronics reviewers who always give 5 stars cluster together. Linear Regression can access the coordinates of each point in that space but has limited capacity to exploit the proximity structure. Acting on cluster membership requires non-linear decision boundaries, which LR does not have. The information is partially accessible, but LR's functional form cannot fully leverage it.

Tree models: benefit conditional on regularisation

XGBoost and Random Forest showed the most variable results. On Amazon, Random Forest improved by 0.1933 overall, but most of that came from Optuna correcting the default max_depth=None setting, not from embeddings. On Yelp, embeddings made RF worse (1.5831 to 1.7030) before Optuna fixed it.

The pattern is consistent: embeddings plus poor hyperparameters can be worse than OHE plus poor hyperparameters. Dense, information rich features give trees more to memorise.
The model must be properly regularised before it can benefit from denser representations.

ANN: largest benefit, but a different mechanism

The ANN benefited most, but the mechanism is not the same as for classical models. For Linear Regression and XGBoost, Experiment 2 is genuine cross model transfer: they receive pre-computed embeddings from a different architecture and use them as static features. For the ANN, Experiment 2 is representation substitution. Sparse OHE inputs are replaced with learned dense inputs. The ANN was always going to learn embeddings internally. The question is whether those embeddings are shaped by the same architecture that uses them (Experiment 2) or derived from a sparse binary encoding (Experiment 1).

The improvement scaled with how badly the OHE baseline was causing problems.
MovieLens: +0.0005 (the OHE matrix had about 50 columns, manageable). Amazon: +0.0775 (high cardinality user and product IDs poorly served by OHE). Yelp: +0.3299 (1,944 sparse columns caused severe memorisation).

The Summary

Dataset	Best Model	Best RMSE	Where improvement came from
MovieLens	XGBoost (Opt)	0.9274	Small gains across all conditions. Feature ceiling dominates.
Amazon	XGBoost (Opt)	1.3579	nisation (RF: +0.193, XGB: +0.136). Encoding helped (+0.077).

Yelp	LR (Emb)	1.5526	eddings fixed ANN (+0.330). LR generalised best on s data.

Table 4: Best results across all datasets.

Three findings held across all three datasets:

1. LR improvement from embeddings was consistently small, always smaller than ANN improvement on the same dataset by at least an order of magnitude.
2. Tree model benefit from embeddings was conditional on hyperparameter regularisation. On none of the three datasets did embeddings alone improve a tree model without tuning.
3. ANN benefit scaled with baseline overfitting severity, not with dataset size or metadata richness.

One finding varied by dataset: XGBoost won on the larger datasets (MovieLens, Amazon). LR won on the smallest (Yelp, 70K training rows). There is no universally best algorithm. The right choice depends on how much data you have.

What is Next

The most impactful missing signal across all three datasets is sequential user history. None of the models here know what a user rated last week, whether their preferences are shifting, or how active they have been recently. Sequence models (GRU4Rec, SASRec) are the natural next step.

The study will also be extended with multi seed variance analysis. All current results are single runs with seed=42. Running each condition 3-5 times and reporting mean plus standard deviation would put the comparisons on firmer statistical ground.

Datasets: MovieLens 20M, Amazon Product Reviews (Kaggle), Yelp Academic Dataset (Kaggle). All publicly available for research.